

CSE 331: MICROPROCESSOR INTERFACING & Embedded Systems

Lecture 3: The ARM Architecture

- **Founded in November 1990**
 - Spun out of Acorn Computers
- **Designs the ARM range of RISC processor cores**
- **Licenses ARM core designs to semiconductor partners who fabricate and sell to their customers.**
 - ARM does not fabricate silicon itself
- **Also develop technologies to assist with the design-in of the ARM architecture**
 - Software tools, boards, debug hardware, application software, bus architectures, peripherals etc



ATAP Partners



Tools Partners



ARM Powered Products



Intellectual Property

- **ARM provides hard and soft views to licencees**
 - RTL and synthesis flows
 - GDSII layout
- **Licencees have the right to use hard or soft views of the IP**
 - soft views include gate level netlists
 - hard views are DSMs
- **OEMs must use hard views**
 - to protect ARM IP

An ARM (Advanced RISC Machine) processor is a type of microprocessor based on the Reduced Instruction Set Computing (RISC) architecture, which emphasizes simplicity and efficiency in instruction execution. ARM processors are widely used in mobile devices, embedded systems, and increasingly in other sectors like servers and personal computers.

Key Features of ARM Processors

1. RISC Architecture:

- ARM processors use a simplified instruction set compared to Complex Instruction Set Computing (CISC) processors like x86. This makes ARM processors more efficient, as they require fewer transistors and consume less power.

2. Power Efficiency:

- ARM processors are designed for low power consumption, making them ideal for battery-powered devices such as smartphones, tablets, and IoT devices.

3. Scalability:

- ARM architectures support a wide range of configurations, from small microcontrollers (e.g., ARM Cortex-M) to high-performance cores for data centers (e.g., ARM Cortex-A series).

4. Load/Store Architecture:

- ARM processors use a load/store architecture where operations are performed only on data in registers, not directly on memory. This reduces complexity and increases speed.

5. Pipeline Processing:

- ARM processors use pipelining to execute multiple instructions simultaneously, improving throughput.

6. Thumb and Thumb-2 Instruction Sets:

- ARM supports compressed instruction sets (Thumb) for reducing code size, which is beneficial for memory-constrained systems.

7. Multicore Support:

- Modern ARM processors often include multicore designs to handle complex computing tasks more efficiently.

Variants of ARM Processors

1.Cortex-M Series:

- Designed for microcontrollers and IoT applications.
- Used in low-power devices like smart sensors and home automation.

2.Cortex-R Series:

- Focused on real-time processing for automotive, industrial control, and safety-critical applications.

3.Cortex-A Series:

- High-performance processors for smartphones, tablets, and servers.
- Features support for advanced computing tasks, multimedia processing, and artificial intelligence.

4.ARMv8 and ARMv9 Architectures:

- Introduced 64-bit support (ARMv8), improving performance for modern applications.
- ARMv9 focuses on enhanced security, AI, and high-performance computing.

Applications of ARM Processors

1.Mobile Devices:

- Dominates the smartphone market due to high efficiency and integration.
- Found in processors like Qualcomm Snapdragon, Apple Silicon, and Samsung Exynos.

2.Embedded Systems:

- Widely used in IoT devices, automotive systems, medical devices, and home appliances.

3.Data Centers and Servers:

- ARM-based processors like AWS Graviton and Ampere Altra are becoming popular for their energy efficiency.

4.Laptops and PCs:

- Apple's M1, M2, and subsequent processors have set new benchmarks in personal computing with ARM architecture.

5.Gaming Consoles and Graphics:

- ARM is used in portable gaming devices and as part of SoCs (System on Chip) for gaming systems.

Advantages of ARM Processors

1. Energy Efficiency:

- Optimal for devices requiring long battery life.

2. Flexibility and Customizability:

- ARM licenses its architecture to manufacturers, allowing custom implementations.

3. Cost-Effectiveness:

- Reduced complexity translates to lower manufacturing costs.

4. Wide Ecosystem:

- Supported by extensive development tools, software libraries, and community resources.

Challenges and Limitations

1. Performance in High-End Computing:

- Historically, ARM processors have lagged behind x86 in high-performance computing, though this is changing.

2. Compatibility:

- ARM-based systems require software recompilation for compatibility, which can be a hurdle for legacy applications.

3. Licensing Model:

- ARM's licensing costs may be a barrier for smaller companies.

Future Trends

1. Adoption in Cloud and AI:

- ARM processors are increasingly used for machine learning and cloud computing workloads.

2. Enhanced Security:

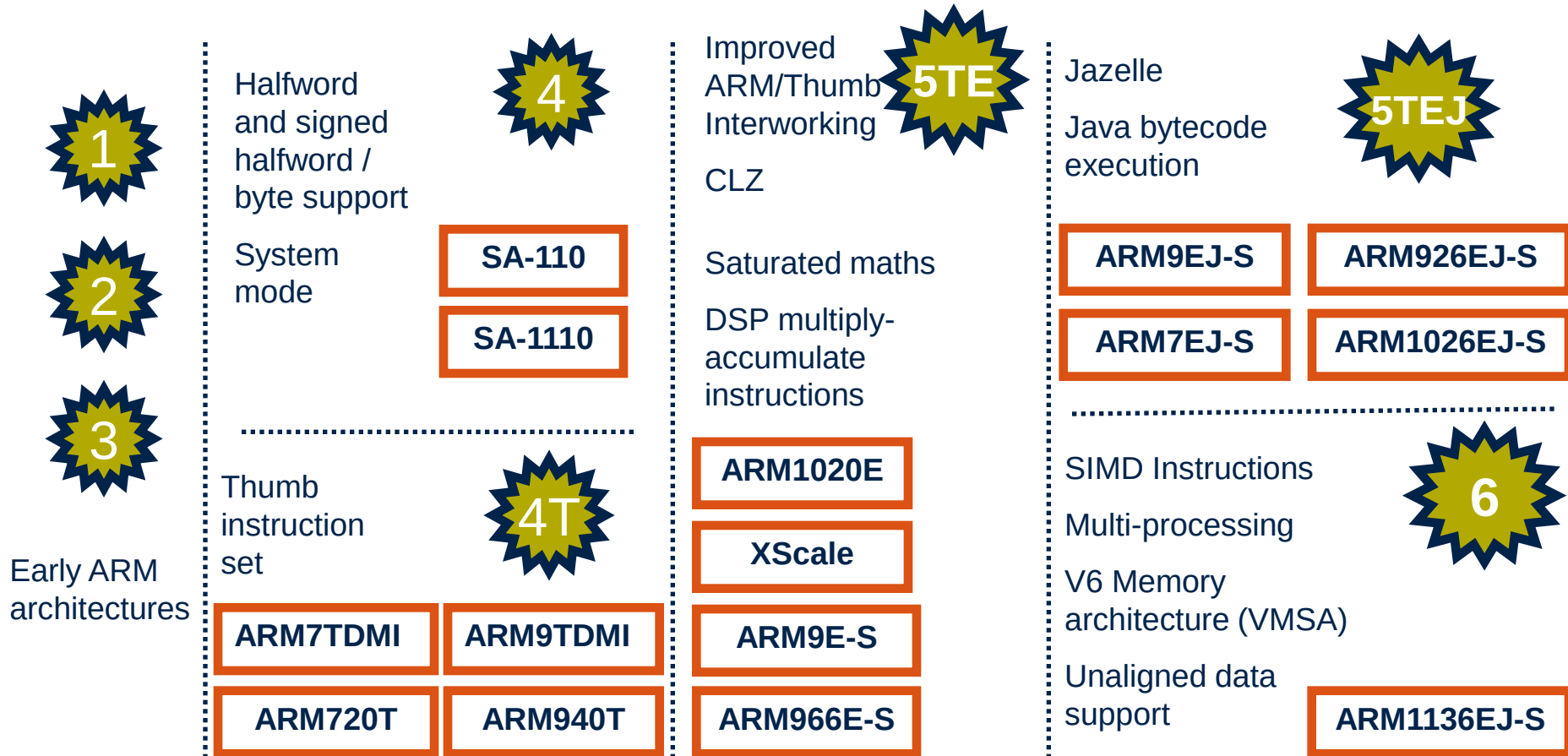
- Features like Memory Tagging Extensions (MTE) in ARMv9 focus on secure computing.

3. Increased Market Penetration:

- ARM's presence in servers and personal computers is growing with advancements in architecture and software support.

In summary, ARM processors excel in efficiency, versatility, and scalability, making them a dominant choice in numerous technology sectors. They continue to evolve, pushing the boundaries of what's possible in modern computing.

Different ARM Architecture



Previous slide outlines the evolution of ARM architectures and highlights key features introduced in different versions:

1. Early ARM Architectures (ARMv1 to ARMv3):

- Introduced halfword and signed halfword/byte support.
- Added a system mode for better control and functionality.

2. Thumb Instruction Set (ARMv4T):

- Added Thumb instruction set for code size reduction and efficiency.
- Key processors: ARM7TDMI, ARM9TDMI.

3. Enhanced Features in ARMv4 and ARMv5TE:

- ARMv4 (e.g., SA-110/SA-1110): Improved ARM/Thumb interworking.
- ARMv5TE: Introduced CLZ (Count Leading Zeros) instruction, DSP (Digital Signal Processing) multiply-accumulate operations, and support for saturated math.
- Key processors: ARM1020E, XScale.

4. Java Bytecode Execution with Jazelle (ARMv5TEJ) :

- Added Java bytecode execution capability with Jazelle.
- Key processors: ARM9EJ-S, ARM926EJ-S, ARM1026EJ-S.

5. Advanced Features in ARMv6:

- Introduced SIMD (Single Instruction Multiple Data) instructions.
- Added multi-processing, enhanced memory architecture (VMSA), and unaligned data support.
- Key processor: ARM1136EJ-S.

This progression demonstrates the focus on efficiency, multimedia capability, and expanding application areas like DSP and Java-based systems.

Architectures after ARMv6

1. ARMv7 (2009) :

- Three profiles introduced:
 - A-profile: Application processors for high-performance systems (e.g., smartphones, tablets).
 - R-profile: Real-time processors for automotive and industrial use.
 - M-profile: Microcontrollers for low-power and embedded applications.
- Features: NEON SIMD, Thumb-2 instruction set, virtualization support.
- Key processors: Cortex-A9, Cortex-M3.

2. ARMv8 (2011) :

- Introduced 64-bit support (AArch64) alongside 32-bit support (AArch32).
- Enhanced cryptography extensions and virtualization.
- Key processors: Cortex-A53, Cortex-A72, Cortex-M33.

Architectures after ARMv6

3. ARMv8.1 to ARMv8.7 (2015–2021):

- Incremental updates to ARMv8.
- Enhanced memory consistency, pointer authentication, and machine learning optimizations.
- Support for features like Scalable Vector Extension (SVE) for HPC workloads.

4. ARMv9 (2021):

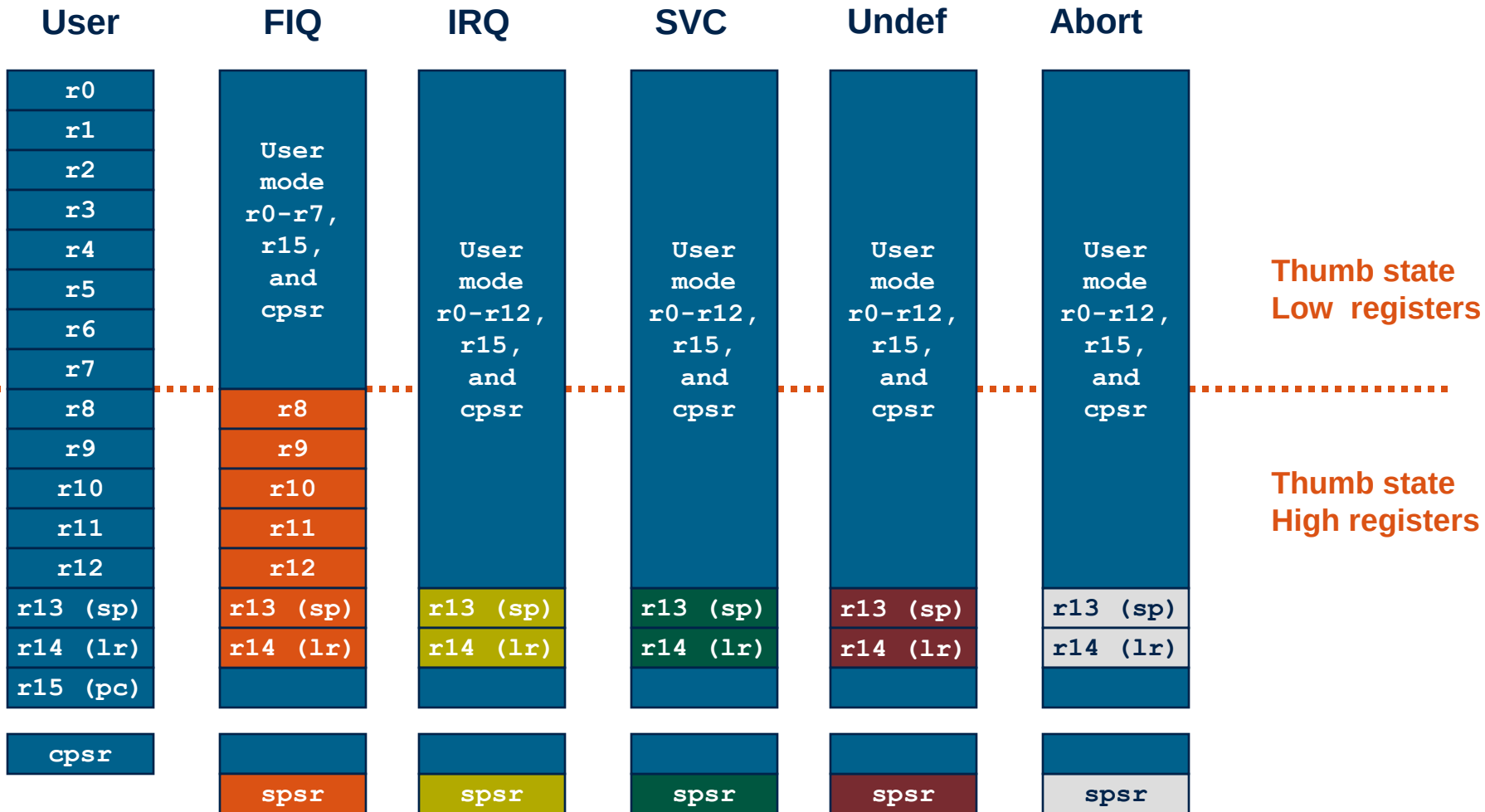
- Focused on security, AI, and high-performance computing.
- Introduced Confidential Compute Architecture (CCA) for data security.
- Advanced support for machine learning and digital signal processing with SVE2 (Scalable Vector Extensions 2).
- Key processors: Cortex-X2, Cortex-A510, Cortex-M85.

Data Sizes and Instruction Sets

- The ARM is a 32-bit architecture.
- When used in relation to the ARM:
 - **Byte** means 8 bits
 - **Halfword** means 16 bits (two bytes)
 - **Word** means 32 bits (four bytes)
- Most ARM's implement two instruction sets
 - 32-bit ARM Instruction Set
 - 16-bit Thumb Instruction Set
- Jazelle cores can also execute Java bytecode

- **The ARM has seven basic operating modes:**
 - **User** : unprivileged mode under which most tasks run
 - **FIQ** : entered when a high priority (fast) interrupt is raised
 - **IRQ** : entered when a low priority (normal) interrupt is raised
 - **Supervisor** : entered on reset and when a Software Interrupt instruction is executed
 - **Abort** : used to handle memory access violations
 - **Undef** : used to handle undefined instructions
 - **System** : privileged mode using the same registers as user mode

Register Organization Summary



Note: System mode uses the User mode register set

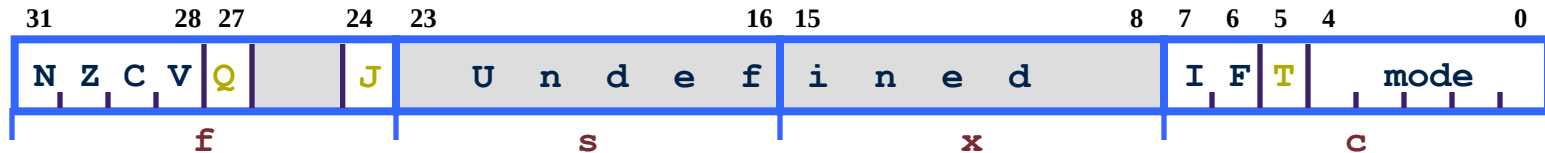
The Registers

- **ARM has 37 registers all of which are 32-bits long.**
 - 1 dedicated program counter
 - 1 dedicated current program status register
 - 5 dedicated saved program status registers
 - 30 general purpose registers
- **The current processor mode governs which of several banks is accessible. Each mode can access**
 - a particular set of **r0-r12** registers
 - a particular **r13** (the stack pointer, **sp**) and **r14** (the link register, **lr**)
 - the program counter, **r15** (**pc**)
 - the current program status register, **cpsr**

Privileged modes (except System) can also access

- a particular **spsr** (saved program status register)

Program Status Registers



■ Condition code flags

- N = **N**egative result from ALU
- Z = **Z**ero result from ALU
- C = ALU operation **C**arried out
- V = ALU operation o**V**erflowed

■ Sticky Overflow flag - Q flag

- Architecture 5TE/J only
- Indicates if saturation has occurred

■ J bit

- Architecture 5TEJ only
- J = 1: Processor in Jazelle state

■ Interrupt Disable bits.

- I = 1: Disables the IRQ.
- F = 1: Disables the FIQ.

■ T Bit

- Architecture xT only
- T = 0: Processor in ARM state
- T = 1: Processor in Thumb state

■ Mode bits

- Specify the processor mode

■ **When the processor is executing in ARM state:**

- All instructions are 32 bits wide
- All instructions must be word aligned
- Therefore the **pc** value is stored in bits [31:2] with bits [1:0] undefined (as instruction cannot be halfword or byte aligned).

■ **When the processor is executing in Thumb state:**

- All instructions are 16 bits wide
- All instructions must be halfword aligned
- Therefore the **pc** value is stored in bits [31:1] with bit [0] undefined (as instruction cannot be byte aligned).

■ **When the processor is executing in Jazelle state:**

- All instructions are 8 bits wide
- Processor performs a word access to read 4 instructions at once

Exception Handling

- **When an exception occurs, the ARM:**
 - Copies CPSR into SPSR_<mode>
 - Sets appropriate CPSR bits
 - Change to ARM state
 - Change to exception mode
 - Disable interrupts (if appropriate)
 - Stores the return address in LR_<mode>
 - Sets PC to vector address
- **To return, exception handler needs to:**
 - Restore CPSR from SPSR_<mode>
 - Restore PC from LR_<mode>

This can only be done in ARM state.

	⋮
0x1C	FIQ
0x18	IRQ
0x14	(Reserved)
0x10	Data Abort
0x0C	Prefetch Abort
0x08	Software Interrupt
0x04	Undefined Instruction
0x00	Reset

Vector Table

Vector table can be at
0xFFFF0000 on ARM720T
and on ARM9/10 family devices